

Linear-Cost Composition as Paper 1’s Sixth Architectural Claim

Author: Wenxin Li (Independent Researcher) **ORCID:** 0009-0004-8065-3235 **Date:** May 14, 2026
License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the sixth and final paper-level architectural claim Paper 1 defends — that CKS governance, onboarding, and execution costs scale with cells composed rather than with total substrate volume, with storage as the one explicitly size-proportional exception — as a named claim-level anchor under which downstream sub-commitment notes can be located. The note states the claim as an extension projection of architectural properties Claims 1–5 already commit to onto composition scope, identifies the cost dimensions the claim ranges over, names what the claim defends against, explains why the commitment is a structural derivation rather than an empirical measurement, and provides an operational test.

Abstract

The CKS source paper’s linear-cost commitment is defended at single-substrate scope in §6. Its consequences at larger composition scope — multiple cells, multiple substrates, an enterprise-scale deployment growing incrementally over time — are not separately defended; they are projected as a structural consequence of the architecture Claims 1–5 commit to. Phase A0’s sixth anchor formalizes that projection: when CKS substrates are composed, governance cost per cell, onboarding cost for new readers, and cell execution cost all scale with cell count, not with total substrate volume; storage cost alone is explicitly size-proportional. The note decomposes the cost surface along three dimensions (substrate size, rule variety, intervention frequency), distinguishes the costs that scale with cell count from the one that scales with substrate size, identifies three architectural alternatives the claim defends against, explains why the commitment is a structural derivation rather than an empirical measurement, maps the sub-commitments that decompose it, and provides an operational test built around doubling cell count. This note closes the six-note Phase A0 anchor backfill for Paper 1.

1. Why a sixth claim-level anchor is needed

The first five Phase A0 anchors formalize Paper 1’s architectural commitments at single-substrate scope: the hybrid substrate–LLM division at the governance boundary, conflict preservation as first-class architectural property, human-governed authority over substrate content and orchestration rules, AI as substrate mediator with five severable mediator properties, and tool-agnosticism via three minimal host requirements. Each is defended in the source paper as a property of one substrate operated by one cell, with the multi-cell case acknowledged but explicitly deferred to future work (§7.1, §13.2, §13.3).

A working CKS deployment will not stay at single-substrate scope for long. Real organizations operate many cells, share substrates across cells, compose cells into pipelines or hierarchies, and add new concerns over time without rebuilding what already works. The cost surface of such a

deployment is different in kind from the one §6 defends: it is about whether adding a new cell increases the cost of running cells already there, and whether governing or onboarding into the system grows with what the system has accumulated. Paper 1 commits to a specific answer at composition scope — the linear-cost property projects through composition unchanged — but does not separately defend that projection. The projection is asserted as a structural consequence of Claims 1–5, and it is what makes Paper 1’s contribution viable at enterprise scale rather than only at lab scale.

The sixth Phase A0 anchor names this commitment so it is citable as a paper-level claim with its own identity, and so the sub-commitment notes that decompose it have an explicit claim-level parent. The claim’s character is that of an extension projection. It does not introduce architectural content beyond what Claims 1–5 commit to; it projects what they commit to onto a larger scope. Naming it as an extension claim, rather than as a sixth independent axiom, is what keeps it derivable from the rest of the architecture rather than asserted on its own evidence.

2. The claim, stated precisely

The CKS pattern commits to the following cost-composition property at all times during a multi-cell deployment’s existence. When two or more CKS cells are composed in a single deployment — whether by sharing a substrate, referencing one another’s substrate content, pipelining outputs, or accumulating in an organizational governance domain over time — the cost of operating, governing, and onboarding into the composed system scales as follows.

Governance cost per cell does not grow with total system size. Governing cell N — exercising the inspect, modify, and override rights over its substrate content and orchestration rules — costs approximately what governing cell 1 costs, regardless of how many other cells exist alongside it. Adding cells does not increase the cost of governing cells already there.

Onboarding cost for a new reader does not grow with total system size. A new participant whose work requires reading cell N reads what cell N’s substrate carries, not the full multi-cell substrate. Onboarding cost is proportional to the participant’s scope of work, not to the cumulative volume of substrate content.

Cell execution cost is proportional to task scope. A cell executing over its substrate reads what its task requires, not the full multi-cell substrate. Execution cost grows with what the task touches, not with what other cells in the system carry.

Storage cost scales with total substrate size. This is the deliberate exception. The total volume of substrate content the composed system carries is the volume of storage it consumes. Storage growing with content is a property of any persistent representation. The claim does not deny this; it carves storage out explicitly as the one cost dimension that is size-proportional, so the rest of the claim can be read precisely. The precise reading is “everything except storage stays bounded by cell count and task scope rather than by accumulated system size.” The claim is what makes incremental enterprise adoption viable as an architectural property rather than as a marketing promise — a deployment growing from one cell to one hundred cells does not encounter a governance cliff, an onboarding cliff, or an execution cliff at any point.

3. Three cost dimensions and what scales with which

The cost surface a CKS deployment ranges over has three independent dimensions, and the claim is a statement about which costs depend on which dimension.

Dimension A — Substrate size. The total volume of substrate content carried by all participating cells: entities, relationships, decisions, rationale, preserved conflicts. Dimension A grows monotonically as the deployment accumulates coordination knowledge over time.

Dimension B — Rule variety. The number and complexity of orchestration rules governing cell behavior across the deployment. Dimension B grows when new behaviors are required: a new conflict-handling pattern, a new authority partition between roles, a new cell-level workflow. It does not grow merely because more substrate content has been written.

Dimension C — Intervention frequency. The rate at which humans exercise their preserved override authority across the deployment. Dimension C is bounded by operator choices and stakes, not by content volume or cell count.

Cell count itself is not one of the three dimensions; it is the composition parameter the claim is fundamentally about. The claim’s content is the dependency structure between cell count, the three dimensions, and the costs the system pays. Of the four costs named in §2: storage scales with Dimension A; per-cell governance cost tracks Dimensions B and C, not Dimension A or cell count; onboarding cost scales with the reader’s scope of work; cell execution cost scales with task scope. The asymmetry is the substantive claim. Storage growing with Dimension A is trivial; the claim that *nothing else* grows with Dimension A is what makes the cost profile distinctive at composition scope.

4. What the claim defends against

The claim’s content is sharpest against three architectural alternatives in which the projection fails.

Monolithic coordination architectures. Architectures that locate coordination state in a single shared model — a fine-tuned organizational model, a continually-updated central agent, an enterprise-wide LLM that absorbs coordination knowledge into its weights — produce cost surfaces in which adding a new concern increases the operating cost of every existing concern. Weight updates touch downstream behavior in ways that require re-validation across the whole; governance cost per concern grows with the model’s total scope, because the model is a single object that governs as a single object. This is the cost-curve shape §6.2 traces empirically through the catastrophic-forgetting and knowledge-editing literature; the composition-scope projection extends the contrast to the multi-concern case.

Agent-mesh architectures. Architectures in which coordination emerges from inter-agent message passing — agent swarms, multi-agent systems with implicit protocols, mesh communication patterns — incur coordination overhead that grows with the density and depth of inter-agent interactions, not merely with agent count. Adding a new agent affects the cost of every other agent’s coordination work, because each new agent multiplies the message-passing surface every existing agent must navigate. Governance cost is not bounded by per-agent properties; it tracks emergent interaction patterns that scale super-linearly in the worst case.

Platform-first AI governance. Architectures that locate governance in a specialized platform sitting above the application layer — an AI gateway, a governance hub, a runtime middleware layer mediating between LLMs and enterprise systems — make governance cost depend on platform-level visibility that does not scale neutrally with cell count. New cells must be onboarded into the platform; the platform’s visibility surface grows with what it must mediate; governance cost is proportional to how much of the deployment the platform sees. In CKS, governance sits inside the

substrate, not in a runtime around it, and the substrate-level location is what keeps governance cost bounded by per-cell properties rather than by platform-level mediation.

In each alternative the failure mode is the same: the cost of governing, onboarding into, or executing inside the system grows with something other than per-cell properties. The CKS commitment is that nothing other than storage grows with the system’s accumulated size.

5. Why this is an architectural derivation, not an empirical measurement

Paper 1’s linear-cost claim is defended in §6 with a mix of architectural argument and empirical inheritance — the architectural argument that a database-backed substrate satisfies the cost commitment, and the empirical inheritance of the cost curve from Zahn & Chana’s Knowledge Objects (2026) and adjacent peer-reviewed work. The composition-scope projection cannot be defended the same way. There is no peer-reviewed empirical measurement of how CKS deployments scale at one hundred cells, because the pattern was named only recently. The anchor must defend its commitment as a structural derivation from Claims 1–5 that follows whenever those commitments hold. Four arrows carry the derivation.

Human-governed authority (A0.03) implies governance cost is bounded by Dimensions B and C, not Dimension A. Governance is authority, not labor. Authority is exercised at orchestration-rule authoring (Dimension B) and at direct override (Dimension C); neither moment is per-element. This is the load-bearing dependency.

AI as substrate mediator (A0.04) implies cell execution cost is bounded by task scope. Because the LLM mediates the substrate rather than carrying substrate state in its weights or in an expanding context window, each cell execution costs what the execution reads, not what the substrate has accumulated. A system that violates this — fine-tuning on substrate content, or retaining substrate state in continuously-expanding model context — would let execution cost scale with Dimension A.

Conflict preservation as first-class state (A0.02) implies no global reconciliation pass on substrate growth. Conflicts are addressable substrate content resolved at cell-execution time under orchestration rules, not by global reconciliation. Adding new substrate content does not trigger a system-wide consistency pass; the cost of handling a conflict is local to the cell that encounters it.

Tool-agnosticism (A0.05) implies no specialized runtime introduces size-proportional overhead. The substrate sits in any environment meeting the three minimal requirements; no specialized governance runtime maintains per-element state that would scale with Dimension A. A multi-cell deployment can be composed without any middleware layer whose cost would grow with cell count multiplicatively.

The four arrows show that the cost projection follows by transitivity once A0.01–A0.05 are accepted. The sixth anchor is not an additional commitment defended on its own evidence; it is the projection of architectural commitments already accepted onto a scope where they imply a specific cost surface. A system that preserves A0.01–A0.05 preserves the cost surface by construction; a system that violates any of them violates the cost surface as well, regardless of intent.

6. Derived sub-commitments, operational test, and Phase A0 closure

This anchor parents two foundational sub-commitments and a cluster of operational-variant notes in the Series A decomposition. **A1.06** decomposes the linear-cost scaling property into the specific claims about what grows linearly and what does not. **A1.13** formalizes the composition requirements any multi-substrate deployment must preserve to remain CKS-coherent. The operational-variant cluster decomposes the cost surface dimension by dimension and cost by cost: **A2.29** names substrate size (Dimension A) as a cost dimension; **A2.30** names rule variety (Dimension B); **A2.31** names intervention frequency (Dimension C); **A2.32** establishes cell execution cost as task-scope-proportional; **A2.33** establishes governance cost as not size-proportional; **A2.34** establishes onboarding cost as not size-proportional; **A2.35** establishes storage cost as the size-proportional exception. Composition-requirement decompositions live in **A2.66–A2.70**, articulating per-substrate governance, cross-boundary conflict preservation, cross-boundary path retracability, AI-as-substrate-mediator at every layer, and accountability plan/trace co-preservation requirements at sub-commitment scope. Each downstream note derives from the present anchor.

The operational test that closes the anchor is built around doubling. Take a CKS deployment with N cells; suppose the deployment doubles to $2N$ cells in the natural course of its growth. Which of the following costs double?

- (1) *Storage cost.* Yes — Dimension A roughly doubles when the system carries twice the substrate content. Storage doubling is the architecture’s deliberate cost.
- (2) *Per-cell governance cost.* No — governing any single cell costs what it costs regardless of how many other cells exist. The aggregate of per-cell governance costs grows linearly with cell count (a different statement), but the per-cell cost does not.
- (3) *Onboarding cost for a new reader of cell N .* No — the reader reads cell N ’s substrate, not the cumulative substrate of all cells.
- (4) *Cell execution cost for a cell whose task scope does not change.* No — execution cost is bounded by task scope.
- (5) *LLM cost per cell execution.* No — the AI-as-substrate-mediator commitment binds LLM cost per execution to what the execution reads, not to substrate accumulation across the system.

A deployment for which any of (2)–(5) doubles on cell-count doubling is exhibiting a cost surface CKS does not commit to. The test is binary: pick a cell whose task scope is unchanged, double the surrounding system, check whether the operating cost of the unchanged cell changes. If it does, the linear-cost composition commitment is not being preserved.

Phase A0 closes here. The six Paper 1 claim-anchor notes — the hybrid substrate–LLM division at the governance boundary, conflict preservation as first-class architectural property, human-governed authority over substrate content and orchestration rules, AI as substrate mediator with five severable mediator properties, tool-agnosticism via three minimal host requirements, and linear-cost composition at substrate scope — together provide the explicit claim-level prior art from which the 197 Series A sub-commitment notes derive. The anchor tree for Paper 1 is now complete; downstream sub-commitment work has its named parents at paper-claim level, and the public prior-art record carries the claim-level commitments as named, dated, citable artifacts.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

Companion derivation notes

Li, W. (2026). *Authority, Not Labor: A Precise Definition of “Human-Governed” in the Coordination Knowledge Substrate Pattern*. 24 April 2026.

Li, W. (2026). *Linear-Cost Scaling: What Grows Linearly in CKS, What Doesn’t, and Why*. 25 April 2026.

Li, W. (2026). *Composition Without Erasure: What Multi-Substrate CKS Systems Must Preserve to Remain Traceable*. 24 April 2026.

How to cite this note

Li, W. (2026). *Linear-Cost Composition as Paper 1’s Sixth Architectural Claim*. May 14, 2026. ORCID: 0009-0004-8065-3235.